SMOOTHIE SHOP E-COMMERCE

Complete Project Documentation & Technical Architecture

Original Frontend by Muhammad Faizan Mustafa | Full-Stack & Admin Upgrades

1. PROJECT OVERVIEW & SYSTEM SCOPE

Independent Full-Stack Smoothie E-Commerce Application

Smoothie Shop is a modern, standalone full-stack e-commerce web application designed for browsing, customizing, ordering handcrafted fresh fruit smoothies, and administering store inventory and customer orders in real time. The platform pairs a responsive React 19 client storefront with a robust Node.js/Express backend, persistent MongoDB database, and an authenticated management dashboard.

2. WHAT YOU BUILT (ORIGINAL DEVELOPER ARCHITECTURE)

Authored by Muhammad Faizan Mustafa

A. Frontend React 19 SPA Architecture

- **Home Showcase (Home.jsx):** Hero slideshow carousel featuring 4 rotating banner slides (Berry Bliss, Mango Glow, Strawberry Freshness, Tropical Energy) with manual arrow and dot controls, accompanied by a 4-column responsive product card grid.
- **Product Details View (ProductDetails.jsx):** Rich product view showing short taglines, comprehensive nutritional descriptions, ingredient highlights, dynamic pricing, and sequential Previous/Next smoothie navigation.
- **Shopping Cart Management (Cart.jsx):** Full cart overview featuring product thumbnails, quantity increment/decrement controls, item removal, real-time subtotal calculations, and responsive layout.
- **Validated Checkout Flow (Checkout.jsx):** End-to-end checkout form built with React Hook Form and Zod schema validation ensuring valid credit card format (xxxx-xxxx-xxxx-xxxx), shipping addresses, and customer emails.
- **Developer Biography (About.jsx):** Portfolio section showcasing developer avatar, 'Behind the blend' bio, and an interactive contact inquiry form.
- **Interactive Chatbot (ChatBot.jsx):** Rule-based keyword matching customer service assistant with predefined answers for product flavors, shipping times, and ordering guidelines.
- **404 Error Routing (Notfound.jsx):** Clean fallback view handling unknown routes.

B. State Management & Authentication

- **CartContext (CartContext.jsx):** Centralized React context provider supplying cart item state, live badge counter for the navbar, and quantity mutation actions.
- **AuthProvider (AuthProvider.jsx):** Client-side authentication provider handling session state and persistence in browser localStorage.
- **LoginModal (LoginModal.jsx):** Interactive modal dialog prompting customers to sign in before purchasing smoothies.
- **Asset Library & Styling:** 10 high-resolution smoothie image assets (Berry, Mango, Tropical, Strawberry, Banana, Pineapple, Peach, Green, Chocolate, Apple) and modular CSS stylesheets.

3. WHAT WAS ADDED & UPGRADED

Backend, Database Integration, Admin Console, and Device Upload

A. Strict Code Preservation & Language Standard

- **Zero Destructive Changes:** 100% of the original codebase, components, logic, and styling were strictly preserved without altering user-written workflows.
- **100% Pure JavaScript:** All newly created files and backend modules were written in pure JavaScript (ES Modules, JSX, zero TypeScript).
- **Clean Environment Routing:** Normalized Vite config and BrowserRouter by removing legacy GitHub Pages subpaths (/Smothie-SHop/) so the application serves natively at root.

B. Full-Stack Express Backend (server.js)

- **Unified Server Architecture:** Built an Express 5 server mounted with Vite development middleware (app.use(vite.middlewares)) on port 3000 (host 0.0.0.0).
- **High-Capacity Body Parsers:** Configured 50MB payload limits for JSON and URL-encoded requests to support high-resolution local device photo uploads.
- **Configuration Template (.env.example):** Documented PORT and MONGODB_URI variables for simple environment bootstrapping.

C. MongoDB & Mongoose Database Integration

- **Mongoose Document Models:** Created schemas for Product (id, title, price, thumbnail, shortDescription, description, category, inStock, timestamps) and Order (customer, items, totalAmount, status, timestamps).
- **Resilient Failover Architecture:** If MongoDB is connected, operations read/write to MongoDB. If offline or unconfigured, the server transparently operates in an in-memory & localStorage sync mode so the app never freezes or crashes.
- **Automated Seeding:** Auto-populates MongoDB with the 10 initial smoothie products on first connection.

D. Complete RESTful API Endpoints (/api/*)

- **GET /api/products:** Retrieves complete catalog of available smoothies.
- **POST /api/products:** Creates a new smoothie product with validation, category, and pricing.
- **PUT /api/products/:id:** Updates pricing, descriptions, titles, and in-stock inventory status.
- **DELETE /api/products/:id:** Deletes a smoothie from the product catalog.
- **POST /api/products/reset:** One-click restore of the original 10 smoothie catalog items.
- **GET & POST /api/orders:** Receives checkout orders and supplies real-time order history.
- **GET /api/health:** Live status probe reporting MongoDB connectivity and product counts.

E. Admin Management Console (/admin)

- **Protected Access:** Gated using the existing AuthProvider with fixed admin credentials.
- **Product & Pricing Manager:** Live table editor to adjust smoothie prices, edit descriptions, toggle stock availability, or delete items.
- **Customer Orders Hub:** Real-time dashboard displaying buyer shipping addresses, ordered items, and order revenue.
- **Database Status Pill:** Visual indicator displaying MongoDB Connected vs. Store Mode.

4. LOCAL DEVICE IMAGE UPLOAD & DEPLOYMENT CONFIGURATION

Media Processing and Hosting Fixes

A. Local Device Image Upload Engine

- **3 Image Sourcing Modes:** Allows store managers to pick from (1) Store Fruit Presets, (2) Local Device Upload, or (3) External Web Image URLs.
- **Interactive File Dropzone:** Admins can browse or drag-and-drop photos directly from their phone, tablet, or laptop (PNG, JPG, WebP, SVG).
- **Client-Side FileReader:** Converts device photos to Base64 data URLs client-side, showing instant image previews with file details and file-reselection controls.
- **Full-Stack Persistence:** Data URLs are processed through the 50MB Express body parser and stored in MongoDB / localStorage for storefront rendering.

B. Vercel SPA Deployment & 404 Refresh Fix (vercel.json)

- **Problem Diagnosed:** When refreshing on /admin or /checkout on Vercel, the host looked for static HTML files (e.g. admin.html) and returned Vercel 404 NOT_FOUND.
- **vercel.json Configuration:** Created root vercel.json with wildcard rewrite ({ 'source': '/(.*)', 'destination': '/index.html' }), instructing Vercel to route all direct page visits and browser refreshes to index.html for React Router.

C. Store Owner Password Management

- **Owner Password Control:** The store owner can update their login password directly inside the Admin Console via the 'Change Password' modal.
- **Persistent Verification:** Password changes are validated against the current password and saved both in local storage and MongoDB for full-stack persistence.

5. TECHNICAL ARCHITECTURE & QUICK REFERENCE

Platform Matrix and Endpoints

LAYER / COMPONENT	TECHNOLOGY STACK	PURPOSE & ROLE
Frontend Framework	React 19 + React Router 7	Client-side SPA UI rendering and routing
Build Engine	Vite 8	Development server & production bundling
Backend Server	Node.js 22 + Express 5	Unified REST API & static asset serving
Database Layer	MongoDB 6+ & Mongoose 9	Document storage with in-memory failover
Form Validation	React Hook Form 7 + Zod 4	Strict schema validation on checkout & forms
Image Processing	FileReader API + 50MB Buffer	Local photo upload & Base64 encoding
Deployment Config	vercel.json SPA Rewrites	Prevents 404 errors on browser page refreshes

QUICK REFERENCE & CREDENTIALS

- Admin URL: /admin | Storefront URL: /
- Admin Credentials: Username: admin | Password: admin (Customizable by owner)